



武汉理工大学  
WUHAN UNIVERSITY OF TECHNOLOGY

# 多智能体交易竞技场：面向股票市场的语言驱动交易系统

课程名称：企业级应用软件设计与开发

项目名称：多智能体交易竞技场：

面向股票市场的语言驱动交易系统

方向：Agentic AI 原生开发

学号：2025302910

姓名：陶倩

专业：计算机技术

指导老师：戚欣

提交日期：2026 年 6 月 10 日

二〇二六年六月



# 目录

- 1. 选题背景与设计思想 .....5
  - 1.1. 问题定义 ..... 5
  - 1.2. 现有方案不足 ..... 5
  - 1.3. 项目价值 ..... 6
  - 1.4. 技术路线 ..... 6
- 2. Specs 规格文档 .....6
  - 2.1. Product Spec ..... 6
  - 2.2. Architecture Spec .....6
  - 2.3. API Spec ..... 7
  - 2.4. 数据规格 ..... 7
- 3. 系统架构与设计 .....7
  - 3.1. 核心架构图 ..... 7
  - 3.2. Agent 交互流程 .....8
  - 3.3. 数据流设计 ..... 8
- 4. 关键实现与代码展示 .....9
  - 4.1. 后端仿真入口 ..... 9
  - 4.2. 仿真主循环 ..... 9
  - 4.3. Agent 下单约束 .....9
  - 4.4. 配置文件 ..... 10
- 5. 测试与评估 .....10
  - 5.1. 功能测试 ..... 10
  - 5.2. Agent 行为评估 .....10
  - 5.3. Benchmark 对比测试 ..... 10
  - 5.4. Demo 截图/录屏 ..... 11
    - 5.4.1. 参数配置仪表盘 ..... 11
    - 5.4.2. 智能体交易详情页 ..... 12

|                             |    |
|-----------------------------|----|
| 5.4.3. Stockmarket 行情 ..... | 14 |
| 5.4.4. Comparison 对比 .....  | 15 |
| 5.4.5. 系统说明 .....           | 16 |
| 6. 系统升级与扩展 .....            | 16 |
| 6.1. 可扩展架构 .....            | 16 |
| 6.2. 下一阶段计划 .....           | 17 |
| 6.3. AI 能力演进路径 .....        | 17 |
| 6.4. 扩展风险与控制 .....          | 17 |
| 7. 课程总结 .....               | 18 |
| 7.1. 个人收获 .....             | 18 |
| 7.2. 工程思维转变 .....           | 18 |
| 7.3. 对课程的建议 .....           | 18 |
| 7.4. 后续自我提升 .....           | 18 |
| 8. 运行说明 .....               | 18 |
| 8.1. 运行环境 .....             | 19 |
| 8.2. 配置要求 .....             | 19 |
| 9. 总结与展望 .....              | 19 |
| 9.1. 项目总结 .....             | 19 |
| 9.2. 未来展望 .....             | 19 |

# 1. 选题背景与设计思想

## 1.1. 问题定义

本项目旨在构建一套可运行、可观测、可回溯的多智能体股票交易仿真平台。平台以多个智能体（Agent）模拟不同类型投资者，使其在实时市场行情、账户资金、持仓状态、公开信息与市场传闻等多源信息约束下自主生成交易决策。智能体的买卖行为将进一步触发市场订单撮合机制，动态影响股票价格、订单状态及参与者资产收益，从而形成较为完整的股票市场博弈闭环。

## 1.2. 现有方案不足

当前主流股票交易仿真系统多基于固定业务规则、随机策略或传统统计模型构建，仍存在两方面不足：第一，系统通常难以接入新闻、传闻、提示词等自然语言信息，无法充分模拟真实投资者的主观判断和差异化交易风格；第二，单纯依赖大语言模型（LLM）进行推演时，若缺少账户资金、持仓数量、订单簿和交易规则等硬约束，模型容易生成不符合交易逻辑的无效指令，工程落地性不足。

本项目在 Django、Vue 与 StockAgent 框架基础上完成系统原型开发，并进一步围绕规格定义、边界校验、运行说明和可视化演示进行完善，使其从“可运行原型”提升为结构清晰、接口明确、流程可复现的工程化系统。



图 1 传统方案与 Agent Trading Arena 对比

## 1.3. 项目价值

本系统将大语言模型的自然语言理解与决策生成能力引入有状态股票交易场景，并通过资金、持仓、交易规则等多重约束确保交易行为合规有效。系统使用 SQLite 持久化保存交易订单、股票行情、用户账户、智能体记忆和市场传闻等关键数据，便于后续复盘分析与实验对比。前端可视化页面能够展示仿真参数、资产收益曲线、股票行情走势及多智能体交易表现差异，因此系统同时具备教学演示、算法验证和智能体行为研究价值。

## 1.4. 技术路线

系统整体采用前后端分离架构。前端基于 Vue 3、Vite 和 Element Plus 搭建交互页面，并结合 ECharts 完成收益曲线、行情走势和对比指标的可视化展示；后端以 Django 作为 API 服务层，承担接口封装、参数接收和任务调度职责；核心仿真模块基于 StockAgent 实现，封装智能体、股票标的、订单撮合、市场指数等领域逻辑；LLM 调用模块通过兼容式 SDK 封装统一接入不同模型服务；仿真产生的全量数据写入 SQLite 数据库，并由后端接口聚合后返回前端展示。

# 2. Specs 规格文档

## 2.1. Product Spec

本产品定位为面向股票市场的多智能体交易仿真竞技场。用户可在前端配置仿真天数、智能体数量、股票标的数量等基础参数并启动实验；系统自动完成市场初始化、智能体决策、订单创建、订单撮合与数据持久化；仿真结束后，用户可查看单个智能体交易明细、股票行情走势、多智能体资产收益对比及基线策略表现，从而完成交易策略和模型效果验证。

## 2.2. Architecture Spec

系统采用四层分层架构，自上而下依次为展示层、API 接口层、仿真领域层、数据与外部服务层。各层职责边界如下：

- (1) 展示层：基于 Vue 开发前端页面，承担参数配置、数据可视化、结果展示与页面导航等交互工作；
- (2) API 接口层：基于 Django 框架提供统一接口，实现前后端数据交互、参数校验与仿真任务调度；

(3) 仿真领域层：由 `main.py`、`Person.py`、`Stock.py`、`Market.py` 等核心文件组成，负责智能体行为生成、股票管理、订单撮合和市场状态更新等核心业务逻辑；

(4) 数据与外部服务层：采用 SQLite 作为本地持久化数据库，使用 Redis 记录缓存与任务状态，并对接外部大语言模型服务。

## 2.3. API Spec

系统对外提供标准化 REST API 接口，核心接口列表如下：`/api/paraminit`（参数初始化）、`/api/simulation_results`（获取仿真结果）、`/api/stock_data`（查询股票行情）、`/api/agent_comparison`（多智能体数据对比）、`/redis_test`（Redis 连通性测试）。所有接口统一采用固定返回格式，包含 `status`（状态码）、`message`（提示信息）、`data`（业务数据）三个字段，保证接口规范性。

## 2.4. 数据规格

系统通过多类数据表存储核心业务数据，主要数据表及用途如下：

- (1) `active_orders`：存储当前未成交订单及其状态信息；
- (2) `stock`：记录股票基本信息、实时价格、历史行情与价格波动数据；
- (3) `person`：维护智能体身份、初始资产、当前资产与持仓信息；
- (4) `account`：记录账户资金变化、盈亏情况与交易流水；
- (5) `memory`：存储智能体历史交易经验、策略反思与行为记忆；
- (6) `gossip`：记录市场传闻、舆情文本及其对智能体决策的潜在影响。

# 3. 系统架构与设计

## 3.1. 核心架构图

系统核心链路为：Vue 前端 → Django API → StockAgent 仿真核心 → SQLite / LLM / Redis。前端负责交互与图表展示，后端负责接口封装和任务调度，仿真核心负责智能体决策与市场撮合，数据库与缓存组件负责数据持久化、状态记录和结果复盘。

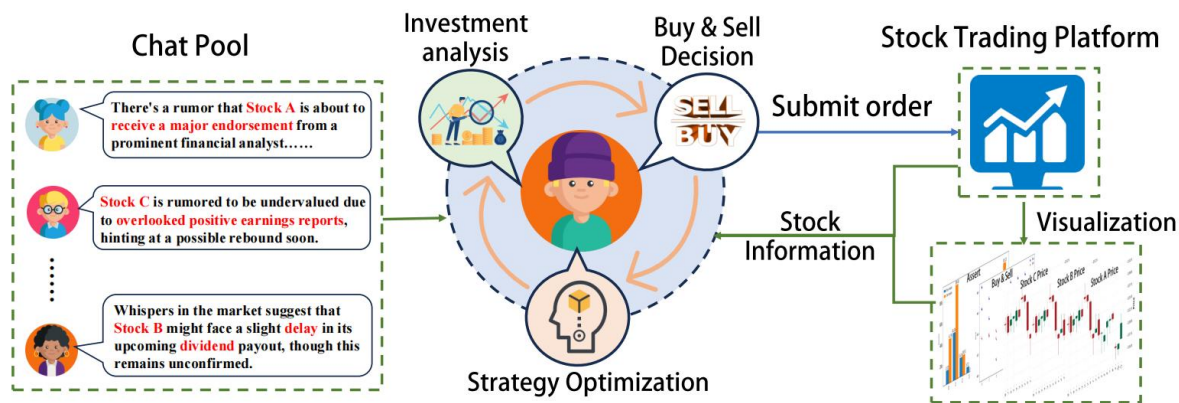


图 2 Agent Trading Arena 的整体框架图

## 3.2. Agent 交互流程

智能体完整运行流程为：系统初始化 → 环境感知 → LLM 决策生成 → 交易订单创建 → 市场订单撮合 → 策略记忆与反思 → 进入下一轮循环。LLM 生成的原始决策指令不会直接执行，而是先经过格式解析、资金与持仓约束校验、交易规则校验等步骤，只有通过校验的订单才会被提交至市场撮合模块。

## 3.3. 数据流设计

数据流从前端参数配置开始。用户提交仿真参数后，后端完成参数校验并更新全局配置，随后启动仿真任务；仿真执行过程中，交易行为、行情变化、智能体资产与持仓状态持续写入 SQLite 数据库；仿真结束后，后端 API 聚合订单表、用户表、股票表等数据，以 JSON 格式返回前端，最终由 ECharts 渲染为收益曲线、行情图 and 对比图表。

表 1 Agent Trading Arena 流程

|          |                                                              |
|----------|--------------------------------------------------------------|
| 核心架构     | Vue 前端 → Django API → StockAgent 仿真核心 → SQLite / LLM / Redis |
| Agent 流程 | 初始化 → 感知 → 决策 → 下单 → 撮合 → 反思                                 |
| 数据流      | 参数配置 → 仿真执行 → 数据库存储 → API 聚合 → 图表展示                          |



## 4. 关键实现与代码展示

### 4.1. 后端仿真入口

后端仿真入口负责接收前端传入的实验参数，完成类型转换与全局配置更新，并调用仿真主程序启动交易模拟。核心代码如下：

```
no_days = int(request.GET.get('No_Days', 11))
num_person = int(request.GET.get('Num_Person', 4))
constant.No_Days = no_days
constant.Num_Person = num_person
constant.Num_Stock = num_stock
overall_test()
```

### 4.2. 仿真主循环

系统以虚拟交易日为基本时间步，按轮次执行市场传闻生成、智能体决策、订单创建与订单撮合等操作。该循环保证每一轮交易都有明确的输入、决策、执行和结果记录。核心代码如下：

```
for virtual_date in range(No_Days):
    generate_gossip(virtual_date, persons, stocks)
    ops = stock_ops(virtual_date, persons, stocks, market_index, iter)
    persons[i].create_order(i, op, virtual_date, iter)
    market.match_order(virtual_date)
```

### 4.3. Agent 下单约束

为避免 LLM 生成不可执行指令，系统对买入和卖出订单设置硬性约束：买入订单必须校验账户可用资金，卖出订单必须校验对应股票持仓数量。只有满足约束条件的订单才会进入撮合流程。约束逻辑如下：

```
if order_type == 'buy' and cash_expense < self.cash:
    submit_order(...)
if order_type == 'sell' and stock_onhold['quantity'] >= quantity:
```

```
submit_order(...)
```

## 4.4. 配置文件

系统使用统一配置文件管理环境变量与全局参数，主要包含仿真基础参数、第三方模型服务密钥以及中间件连接地址等内容：

```
No_Days = 3
Num_Person = 4
Num_Stock = 3
DASHSCOPE_API_KEY=
REDIS_HOST=localhost
DJANGO_SECRET_KEY=
```

## 5. 测试与评估

### 5.1. 功能测试

本项目围绕系统核心功能开展测试，测试范围覆盖参数初始化、仿真任务执行、仿真结果查询、股票行情展示、多智能体数据对比、安全配置检查和文档导航校验等模块。测试重点在于验证功能链路完整性、接口返回稳定性以及异常输入下的边界处理能力。

### 5.2. Agent 行为评估

智能体交易表现从以下四个维度进行评估：

1. 有效性：智能体能够稳定输出买入、卖出、持有三类标准交易指令；
2. 约束性：交易行为严格遵循账户资金、持仓数量和交易规则约束；
3. 多样性：不同智能体能够基于自身策略、记忆和市场信息产生差异化交易行为；
4. 可解释性：依托数据库中的记忆与交易记录，可回溯关键决策的生成过程与执行结果。

### 5.3. Benchmark 对比测试

Benchmark 测试选取经典交易策略作为基线模型，并与本项目的 LLM 智能体进行对比。评估指标包括最终总资产、累计收益率、最大回撤、交易次数、有效订单率和接口响应耗时等，用于从收益表现、风险控制和系统性能三个层面衡量方案效果。

表 2 Arena Trader 在纳斯达克和沪深证券交易所的策略表现

| Strategy                        | NASDAQ       |              |              |              |              | CSI          |              |              |              |              |
|---------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                                 | TR ↑         | Mean ↑       | Std ↓        | WR ↑         | SR ↑         | TR ↑         | Mean ↑       | Std ↓        | WR ↑         | SR ↑         |
| Buy & Hold                      | 6.06         | 0.155        | 0.916        | <b>61.54</b> | 0.169        | 0.73         | 0.004        | 0.403        | 43.15        | 0.010        |
| SMA                             | 0.19         | 0.006        | <u>0.561</u> | 46.15        | 0.011        | -3.45        | -0.014       | <b>0.319</b> | 33.20        | -0.044       |
| ZMR                             | 1.04         | 0.030        | 0.814        | 41.03        | 0.037        | 1.24         | 0.006        | <u>0.361</u> | 41.91        | 0.016        |
| MACD                            | 5.62         | 0.146        | 1.033        | 58.97        | 0.141        | -0.67        | -0.002       | 0.373        | 40.66        | -0.006       |
| StockFormer (Gao et al., 2023)  | 11.23        | 0.281        | <b>0.121</b> | 56.41        | 0.231        | 14.83        | <b>0.636</b> | 1.128        | 41.08        | 0.056        |
| TimesNet (Wu et al., 2023)      | 11.41        | 0.283        | 1.035        | 56.41        | 0.273        | 32.93        | 0.139        | 2.053        | <u>48.55</u> | 0.068        |
| StockMixer (Fan and Shen, 2024) | 14.64        | 0.357        | 1.173        | 56.41        | <u>0.305</u> | 43.00        | 0.153        | 0.907        | 27.80        | <b>0.168</b> |
| ArenaTrader w/ GPT-4o (Ours)    | <b>15.99</b> | <b>0.450</b> | 1.110        | <u>58.97</u> | <b>0.348</b> | <b>48.93</b> | <u>0.176</u> | 1.432        | <b>51.45</b> | <u>0.123</u> |

表 3 Arena Trader 使用不同 LLM 的投资表现。

| Model                       | Avg. Trend | TR (%) ↑       | Mean (%) ↑    | Std (%) ↓     | WR (%) ↑      | SR ↑           | Δ (%) ↑        |
|-----------------------------|------------|----------------|---------------|---------------|---------------|----------------|----------------|
| ArenaTrader w/ Qwen-VL-128k | -0.4632    | 33.3018        | 2.6526        | 1.0150        | <b>100.00</b> | 2.6135         | 33.7650        |
| ArenaTrader w/ GPT-4o-mini  | 5.4722     | 38.6460        | 3.0249        | 1.4235        | <b>100.00</b> | 2.1249         | 33.1738        |
| ArenaTrader w/ Gemini-1.5   | 7.2743     | <u>41.3264</u> | <u>3.1946</u> | <b>0.1588</b> | <b>100.00</b> | <b>20.1128</b> | 34.0521        |
| ArenaTrader w/ Qwen-VL-32k  | -0.5216    | 35.8150        | 3.1213        | 1.6079        | 90.00         | 1.9412         | <u>36.3366</u> |
| ArenaTrader w/ GPT-4o       | 3.5881     | <b>47.6851</b> | <b>3.6095</b> | <u>0.5327</u> | <b>100.00</b> | <u>6.7765</u>  | <b>44.0970</b> |

## 5.4. Demo 截图/录屏

Demo 演示围绕“配置—运行—观察—对比—说明”五个环节展开，主要包括参数配置仪表盘、智能体交易详情页、股票实时行情页、多智能体数据对比页和系统说明页面。通过这些页面可以完整展示系统从实验启动到结果分析的闭环流程。

### 5.4.1. 参数配置仪表盘

参数配置仪表盘是 Demo 的入口页面，主要用于设置仿真天数、智能体数量、股票数量等核心实验参数。用户在该页面完成配置后，可一键启动仿真任务，后端会根据参数初始化市场环境、智能体账户和股票标的。该页面体现了系统的可配置性，便于对不同实验规模和市场设定进行快速复现实验。



图 3 参数配置

### 5.4.2. 智能体交易详情页

智能体交易详情页用于展示单个 Agent 在仿真过程中的资产变化、交易记录和决策结果。页面以折线图展示智能体资产随虚拟交易日的变化趋势，并在下方表格列出对应交易日的买卖动作、股票标的及交易数量。通过该页面可以观察不同智能体在相同市场环境下的行为差异，也便于追踪某一智能体是否存在频繁交易、保守持有或集中买卖等策略特征。



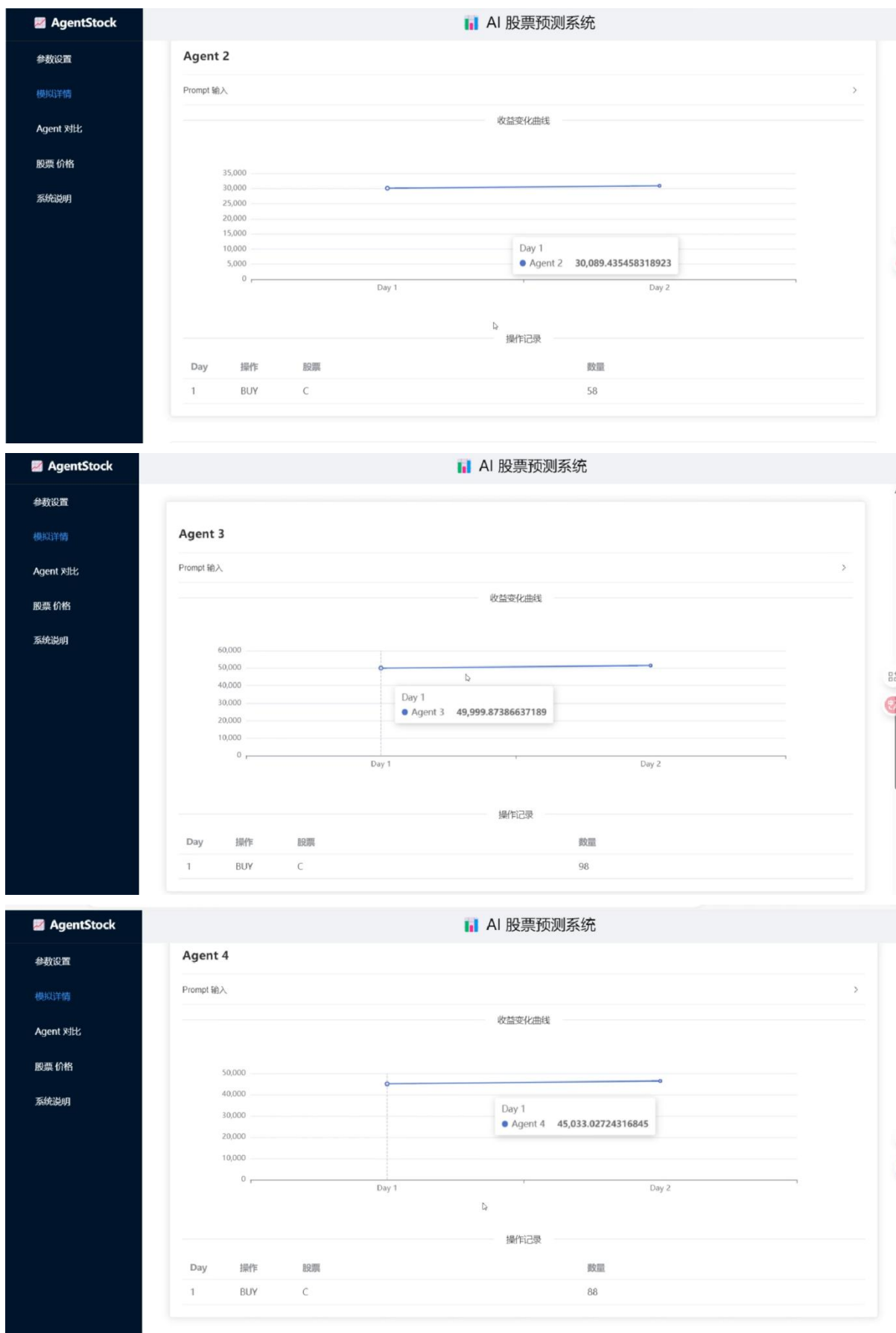
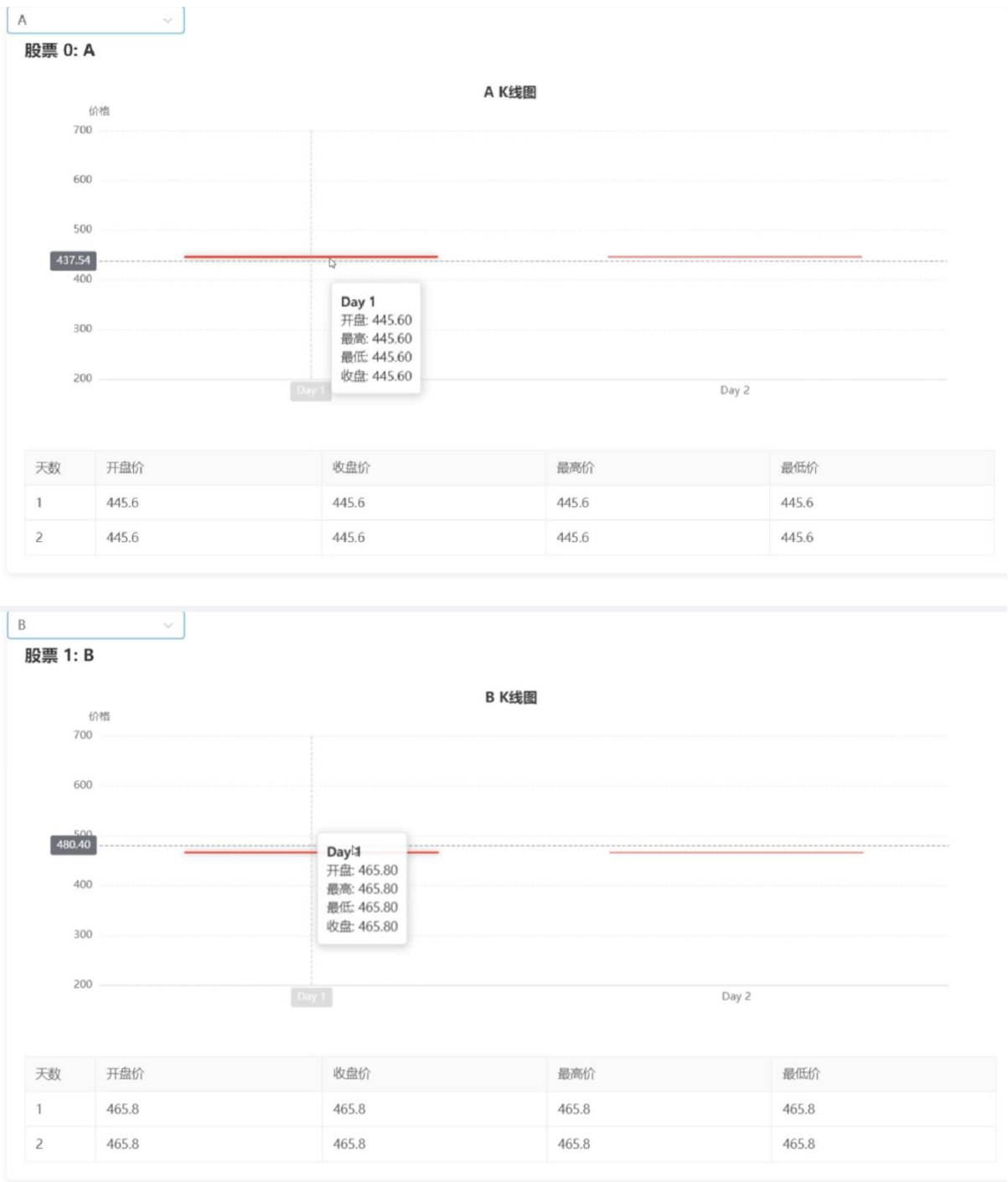


图 4 智能体交易详情

### 5.4.3. StockMarket 行情

StockMarket 行情页面向股票标的维度展示市场变化。每只股票对应独立的行情图和数据表，图表展示开盘价、最高价、最低价、收盘价等关键指标，表格则保留每日行情记录。该页面能够帮助用户判断智能体交易行为是否对市场价格产生影响，并进一步分析股票价格波动与订单撮合结果之间的关系。



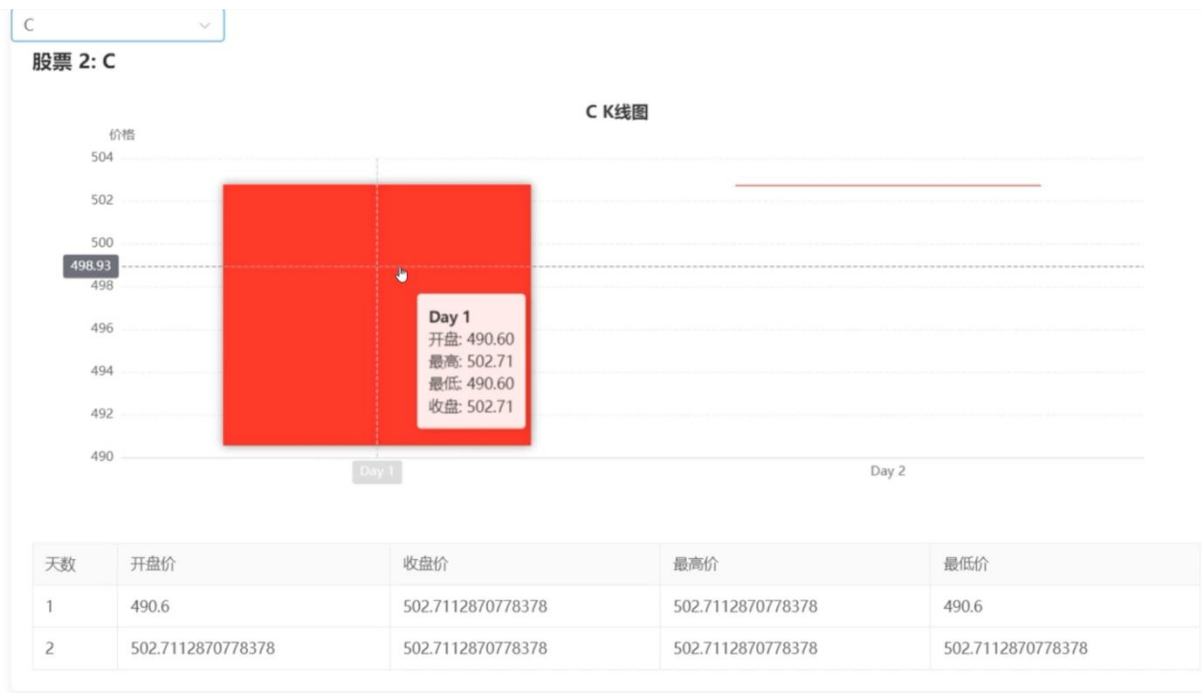
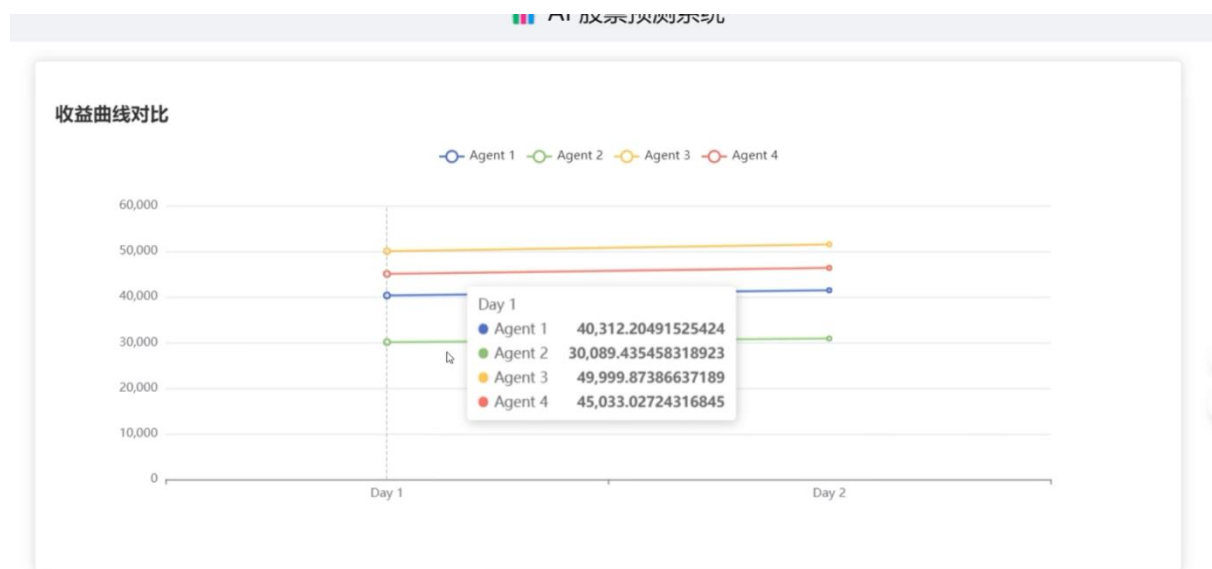


图 5 股票市场行情

#### 5.4.4. Comparison 对比分析

Comparison 对比页面用于横向比较多个智能体的资产收益表现。页面将不同 Agent 的总资产变化绘制在同一坐标系中，并给出最终收益排行，便于直观识别表现较优和较差的智能体。通过该页面可以进一步评估不同提示词、策略设定或模型配置对交易收益的影响，是系统进行 Benchmark 对比和实验复盘的重要入口。



| 最终收益排行   |                    |
|----------|--------------------|
| Agent ID | 最终收益               |
| 3        | 51496.46986904445  |
| 4        | 46353.79757830508  |
| 1        | 41452.22909694915  |
| 2        | 30883.589829152545 |

图 6 多智能体对比

### 5.4.5. 系统说明

系统说明页面对平台定位、核心功能、运行流程和扩展方向进行集中说明，便于首次使用者快速理解系统。该页面将项目目标、仿真流程、参数含义和结果解读方式进行整合，降低了演示和复现实验的使用门槛，也使系统具备更好的课程展示与项目交付完整性。



图 7 系统说明

## 6. 系统升级与扩展

### 6.1. 可扩展架构

后续可将现有同步执行架构升级为“API 网关 + 异步任务队列 + 仿真工作节点 + 结果



数据仓库”的分布式架构。前端提交仿真任务后，系统返回唯一任务编号（task\_id）；仿真工作节点异步执行 LLM 决策与订单撮合逻辑；Redis 实时记录任务运行状态；仿真结果统一写入数据库，支持后续查询、复盘与二次分析。

## 6.2. 下一阶段计划

表 4 下阶段计划

| 迭代阶段 | 核心建设内容                                  |
|------|-----------------------------------------|
| 阶段 1 | 工程化优化：依赖管理、Docker 部署、单元测试与配置规范化         |
| 阶段 2 | 任务管理：task_id 生成、进度监控、历史实验存储与结果复盘        |
| 阶段 3 | Benchmark 面板：规则基线、统计指标与 LLM Agent 自动化对比 |
| 阶段 4 | 能力拓展：多模型、多市场、自定义策略与多智能体协同               |

## 6.3. AI 能力演进路径

AI 能力演进遵循“基础提示词智能体 → 工具调用智能体 → 反思型智能体 → 多智能体协同”的路线。后续将逐步为智能体补充工具调用、长期记忆、策略反思和多角色协作能力，使其能够在更复杂的市场环境中形成持续迭代的交易策略。

## 6.4. 扩展风险与控制

系统扩展过程中可能面临响应延迟、模型输出不稳定、实验不可复现和调用成本上升等风险。相应的控制策略包括：固定随机种子以保证实验可复现；保存提示词版本和模型配置；增加输出格式与交易规则校验；通过任务队列和限流机制控制并发压力；统计大模型 Token 消耗并进行成本监控。

表 5 控制风险方法

|      |                              |
|------|------------------------------|
| 阶段 1 | 工程化：依赖管理、Docker 部署、测试用例与配置模板 |
| 阶段 2 | 任务管理：task_id、进度查询、历史实验与结果复盘  |

|      |                                     |
|------|-------------------------------------|
| 阶段 3 | Benchmark：规则基线、统计指标与 LLM Agent 横向对比 |
| 阶段 4 | 多模型、多市场、多策略与协同智能体扩展                 |

## 7. 课程总结

### 7.1. 个人收获

通过本项目实践，我不再停留于“调用大模型生成文本”的基础应用，而是进一步理解了如何将 LLM 嵌入有状态、强约束、可反馈的复杂业务系统。项目开发过程加深了我对 Agentic AI 工程落地方式的认识，也提升了我对架构分层、接口规范和实验评估的综合理解。

### 7.2. 工程思维转变

本项目促使我的开发思维从“功能跑通即可”转向“可维护、可复现、可评估、可扩展”的工程化思维。完整的软件交付不仅需要可运行代码，也需要清晰的规格文档、合理的架构设计、必要的功能测试、安全配置和运行说明。

### 7.3. 对课程的建议

建议课程后续提供更标准化的课程报告模板，并增加 AI 智能体工程化、AI 开发工具（如 AI IDE）和真实项目协作流程等专项教学案例。同时，可设置中期代码评审和阶段性 Demo 展示环节，以便学生在项目早期获得更明确的实践反馈。

### 7.4. 后续自我提升

后续我将重点提升系统架构设计、自动化测试、模型评估和工程部署能力，并继续深入研究 Agent 工作流，实现从基础提示词调用向工具调用、长期记忆、策略反思和多智能体协作等高阶能力的升级。

## 8. 运行说明

## 8.1. 运行环境

(1)后端服务：进入 `src/AgentStock` 目录，执行命令 `python manage.py runserver` 启动 Django 后端服务；

(2)前端服务：进入 `src/AgentStock/Frontend` 目录，执行命令 `npm run dev` 启动 Vue 前端页面。

## 8.2. 配置要求

若需启用完整 LLM 仿真功能，需要在配置文件中填写对应模型服务密钥，支持 `DASHSCOPE_API_KEY`、`DEEPINFRA_API_KEY`、`GOOGLE_API_KEY`、`OPENAI_API_KEY` 等主流接口配置。若启用 Redis 连通性测试或任务状态缓存功能，还需正确配置 `REDIS_HOST`、`REDIS_PORT`、`REDIS_PASSWORD`、`REDIS_DB` 等参数。

# 9. 总结与展望

## 9.1. 项目总结

本项目完成了一套基于大语言模型的多智能体股票交易仿真平台，整合了智能体决策、市场订单撮合、全量数据存储和 Web 可视化四大核心模块，实现了从参数配置、仿真运行到结果展示的闭环式交易模拟功能，可用于模拟多投资者参与下的市场博弈过程。

## 9.2. 未来展望

未来将继续围绕系统稳定性、实验可复现性和量化分析能力进行优化：一方面完善仿真结果可视化和多组实验横向对比能力，另一方面增强任务管理、日志追踪和模型评估功能，使平台逐步具备更完整的研究与教学支撑能力。